Studio Nx

Backend(FastAPI/Python) 09 — Inquiry 모델 컬럼

2026-10-06

Backend(FastAPI/Python) 09 — `Inquiry` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/inquiry.py`
작업	P1-03 백엔드 공통 — ORM 모델 컬럼 · `docs/DB.md` §3.1 · `studionx/db/01_phase1_account.sql`
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	08 `User` 모델 · 10 `Project` 모델

1. 하는 일

MySQL 의 `inquiries` 테이블(문의)을 파이썬 클래스 `Inquiry` 로 옮겨 적는다. 08 `User` 와 같은 방식이고, 이번에 새로 나온 것은 **다른 테이블을 가리키는 칸(FK)** 과 **긴 글 칸(`TEXT`)** 이다.

새로 나온 것	칸	뜻
FK (외래 키)	`user_id` → `users.user_id`	이 문의를 보낸 회원. 손님이면 비어 있음
`TEXT`	`content` · `answer`	길이 제한이 거의 없는 글 (`VARCHAR` 는 길이를 정해야 함)

2. 언제 쓰나

쓰는 곳	하는 일	칸
P1-12 문의 등록 (home.html Contact 폼)	문의 저장	`user_id`(07 의 결과) · `email` · `title` · `content` · `privacy_agreed_at`
P1-21 관리자 화면 (SQLAdmin)	목록 보기 · 답변	`status` · `answer` · `answered_at`

문의 상태 (`status`)	뜻
`RECEIVED` (기본)	접수됨, 아직 답변 없음
`ANSWERED`	관리자가 답변함
`CLOSED`	처리 끝

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from enum import StrEnum

from sqlalchemy import BigInteger, Enum, FetchedValue, ForeignKey, String, Text, text
from sqlalchemy.dialects.mysql import DATETIME
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`BigInteger` · `Mapped` · `mapped_column` · `Base` 는 원래 있었고, 나머지를 이번에 추가했다. 08 과 같고, **`ForeignKey`** · **`Text`** 두 개가 새로 들어갔다.

import	가져오는 것	쓰는 곳
`ForeignKey`	다른 테이블의 칸을 가리키는 표시	`user_id`
`Text`	SQL 의 `TEXT`	`content` · `answer`
나머지	08 의 표 와 같음	

함수 (클래스 · 칸 선언)

```
class InquiryStatus(StrEnum):
    RECEIVED = "RECEIVED"
    ANSWERED = "ANSWERED"
    CLOSED = "CLOSED"


class Inquiry(Base):
    __tablename__ = "inquiries"

    inquiry_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)

    user_id: Mapped[int | None] = mapped_column(BigInteger, ForeignKey("users.user_id"))
    name: Mapped[str | None] = mapped_column(String(50))
    email: Mapped[str] = mapped_column(String(255))
    title: Mapped[str] = mapped_column(String(200))
    content: Mapped[str] = mapped_column(Text)
    privacy_agreed_at: Mapped[datetime] = mapped_column(DATETIME(fsp=3))
    status: Mapped[InquiryStatus] = mapped_column(
        Enum(InquiryStatus), server_default=InquiryStatus.RECEIVED
    )
    answer: Mapped[str | None] = mapped_column(Text)
    answered_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    updated_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )
```

SQL 과 비교 (08 에 없던 것만)

SQL (`01_phase1_account.sql`)	파이썬
`user_id BIGINT UNSIGNED NULL` + `FOREIGN KEY (user_id) REFERENCES users (user_id)`	`Mapped[int | None]` · `ForeignKey("users.user_id")`
`content TEXT NOT NULL`	`Mapped[str]` · `Text`
`KEY idx_inquiries_status (status)` (검색용 색인)	적지 않음 — 아래 §5

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀

식	뜻
`ForeignKey("users.user_id")`	"이 칸의 값은 `users` 테이블 `user_id` 중 하나"라는 표시. `"테이블.칸"` 글자로 적음
`Mapped[int | None]` + FK	비어 있어도 됨(손님 문의). 값이 있으면 실제 회원 번호여야 함
`Text`	길이를 적지 않음. MySQL `TEXT` 는 약 64KB (한글 약 2만 자)

4. 흐름 (손님 문의 하나를 넣을 때)

1. `Inquiry(email=…, title=…, content=…, privacy_agreed_at=…)` — 4칸만 넣는다.
2. `db.add` → `db.flush()` 로 `INSERT`.
3. DB 가 `inquiry_id` · `status=RECEIVED` · `created_at` 을 채운다. `user_id` 는 비어 있다.
4. 로그인 회원이면 1에서 `user_id=7` 을 함께 넣는다. 없는 번호면 DB 가 거절한다.

5. 왜 이렇게

선택	이유
`user_id` 를 비울 수 있게	손님도 문의할 수 있다 (FR-112). 07 이 `None` 을 주면 그대로 비워 둔다
FK 를 파이썬에도 적음	테이블은 SQL 이 만들지만, 적어 두면 나중에 `relationship`(문의 → 회원)을 붙일 수 있고 코드만 봐도 연결을 알 수 있다
색인(`idx_inquiries_status`)은 적지 않음	색인은 DB 가 검색을 빠르게 하려고 가진 것이다. 파이썬에서 읽고 쓰는 데 필요 없다. 08 도 같다
`content` 최대 2,000자는 여기서 검사하지 않음	칸(`TEXT`)은 2,000자보다 훨씬 크다. 글자 수 검사는 입력 스키마(P1-12, 02 의 422)가 한다
`name` 칸도 적음	폼에서 받지 않는 예비 칸이지만 테이블에 있으므로 모델도 같게 맞춘다
회원 탈퇴 때 문의는 남김	FK 에 `ON DELETE` 가 없다. 탈퇴는 행을 지우지 않고 `status=DELETED` 로 바꾸므로(08) 문의의 `user_id` 도 그대로 남는다

6. 시험

`tests/test_inquiry_model.py` (새로 만듦 · 5개) — 실제 MySQL 과 비교했다. 넣은 행은 끝나면 되돌린다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB 와 같은지	칸 12개 이름 · 순서 · `NULL` 허용 · FK(`user_id` → `users.user_id`)가 실제 테이블과 같음
손님 문의	4칸만 넣으면 `user_id` 비어 있음 · `status` `RECEIVED` · `created_at` 채워짐 · 답변 칸 비어 있음
회원 문의	회원을 만들고 그 번호를 넣으면 저장됨
없는 회원 번호 (999999999)	DB 가 거절 (`IntegrityError`)
2,000자 글	그대로 저장되고 다시 읽힘

`tests/conftest.py` 정리 (모든 모델 시험이 같이 씀)

추가한 것	하는 일
`db`	실제 MySQL 세션. 연결할 수 없으면 시험을 건너뛰고, 끝나면 되돌림(rollback)
`assert_matches_db(모델)`	모델의 칸 이름 · 순서 · `NULL` 허용 · FK 를 실제 테이블과 비교

08 의 `tests/test_user_model.py` 에 있던 같은 코드를 여기로 옮겼고, 08 시험도 이것을 쓰도록 바꿨다. 남은 모델(10 ~)도 이것을 쓴다.

전체 시험 57개 통과, `ruff` 검사 통과.